processing of XML and other attacks, which you can find linked from the deserialization cheat sheet.

Although the API is using a safe JSON parser, it is still trusting the input in other regards. For example, it doesn't check whether the supplied username is less than the 30-character maximum configured in the database schema. What happens you pass in a longer user name?

```
$ curl -d '{"name":"test", "owner":"a really long username
[CA] that is more than 30 characters long"}'
[CA] http://localhost:4567/spaces -i
HTTP/1.1 500 Server Error
Date: Fri, 01 Feb 2019 13:28:22 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.8.v20171121)
{"error":"internal server error"}
```

If you look in the server logs, you see that the database constraint caught the problem:

```
Value too long for column "OWNER VARCHAR(30) NOT NULL"
```

But you shouldn't rely on the database to catch all errors. A database is a valuable asset that your API should be protecting from invalid requests. Sending requests to the database that contain basic errors just ties up resources that you would rather use processing genuine requests. Furthermore, there may be additional constraints that are harder to express in a database schema. For example, you